



CHAM: A Family of Lightweight Block Ciphers for Resource-Constrained Devices

Bonwook Koo^(✉), Dongyoung Roh, Hyeonjin Kim, Younghoon Jung,
Dong-Geon Lee, and Daesung Kwon

National Security Research Institute, Daejeon, Republic of Korea
{bwkoo, dyroh, mikjh, sky1236, guneez, ds_kwon}@nsr.re.kr

Abstract. In this paper, we propose a family of lightweight block ciphers CHAM that has remarkable efficiency on resource-constrained devices. The family consists of three ciphers, CHAM-64/128, CHAM-128/128, and CHAM-128/256 which are of the generalized 4-branch Feistel structure based on ARX (Addition, Rotation, XOR) operations.

In hardware implementations, CHAM requires smaller areas (73% on average) than SIMON [8] through the use of a *stateless-on-the-fly* key schedule which does not require updating a key state. Regarding software performance, it achieves outstanding figures on typical IoT platforms in terms of the balanced performance metrics introduced in earlier works. It shows a level of performance competitive to SPECK [8] mainly due to small memory size required for round keys. According to our cryptanalysis results, CHAM is secure against known attacks.

Keywords: Lightweight block cipher · Stateless-on-the-fly · ARX

1 Introduction

We are in the pervasive computing era. One can interact with many computing devices, such as laptop computers, tablets, smart phone, and even glasses with computing capabilities. Gartner, Inc. forecasts that in 2017, up 31 percent from 2016, 8.4 billion connected devices will be in use worldwide, with the number reaching 20.4 billion by 2020. The development and spread of these computing devices have made human life more convenient and enriching. However, in terms of adverse effects, threats that existed in cyberspace have spread and expanded to our everyday lives. So, it has become crucial to address security issues in relation to highly constrained devices so as to protect our lives and property. The first step in securing pervasive devices is to ensure that all data from them are transferred confidentially. Cryptographic algorithms, especially block ciphers, have been used extensively to perform this role.

Lightweight cryptography is essential for reducing size and cost of computing and communicating devices. So in cryptography, lightweighness has been moved from implementation issues to design consideration. A lot of block

ciphers have been designed for being lightweight in various aspect (for example chip size, code size, power consumption, memory usage, and so on) on various platforms for the past decade. Some of these algorithms have been shunned for various reasons including security problems, but still many algorithms are being discussed, applied or standardized in the real world such as HIGHT [31], PRESENT [6], CLEFIA [44], KATAN/KTANTAN [21], PRINTCIPHER [34], LED [29], PICCOLO [43], PRINCE [19], SIMON/SPECK [8], LEA [30], PRIDE [1], MIDORI [3], SPARX [28], SKINNY [11] and recently proposed GIFT [4].

Among these algorithms, SIMON and SPECK designed by NSA in 2013, are the most focused and evaluated ciphers. They are families of non-S-box based lightweight block ciphers. SIMON is tuned for optimal performance in hardware, while SPECK is designed for optimal performance in software. SPECK is in ARX structure. They are arguably regarded as the best algorithms among previously proposed lightweight block ciphers on hardware and software platforms.

LEA is another ARX cipher designed by Hong et al. in 2013 and suitable for common platforms (mainly 32-bit microcontrollers), but its performance is not so impressive on 8-bit and 16-bit microcontrollers.

Contributions. We have attempted to improve LEA by increasing the level of suitability for resource-constrained environment such as hardware and 8-bit and 16-bit microcontrollers. As a result, we propose a new family of block ciphers, CHAM.

CHAM has the following features:

- CHAM uses an extremely simple key schedule possibly being implemented without updating key states. This feature helps to reduce the number of flip-flops when implementing our ciphers on hardware.
- Numbers of round keys are far fewer than the numbers of rounds, and round functions reuse them iteratively. This reduces the memory size necessary to store the round keys.
- Encryption uses two types of left rotation, by 1 bit and by 8 bits, to minimize the number of operations on 8-bit AVR microcontroller.
- Round indexes are used as round constants instead of random values to save resources while defending against slide attacks [18] and basic rotational attacks [33].

There are many lightweight block ciphers which have the same feature of key schedule, such as HIGHT, KATAN, PRINTCIPHER, LED, PICCOLO, PRINCE, PICARO [41], PRIDE, and KHUDRA [36]. However, some of these ciphers are prone to or fully broken by related-key attacks [22, 23, 25, 32, 37, 54]. Bearing this in mind, we analyze the security of CHAM against various attacks, including differential cryptanalysis and linear cryptanalysis. In particular, we give more attention to security analyses in the related-key model.

By efficiency evaluation and comparison, we draw the following results. On hardware, we implement SIMON and our ciphers by minimizing the area with the IBM 130 nm CMOS generic process library. The results are briefly presented

in Table 1. Table 2 shows a brief comparison with SPECK on the two widely adopted target platforms of Atmega128 and MSP430. CHAM shows efficiency competitive with one of the top-ranked algorithms, SPECK on the platforms. In summary, CHAM can be implemented using smaller area than SIMON in hardware environment and shows a similar level of efficiency to SPECK in 8- and 16-bit software environments.

Table 1. Area-optimized implementation results in gate for CHAM and SIMON.

Algorithm	Library	64/128	128/128	128/256	Ref.
CHAM	IBM 130 nm	665	1,057	1,180	This paper
SIMON	IBM 130 nm	958	1,234	1,782	[8]

Table 2. Software efficiency comparison in *rank* metric [9] (Larger is better).

Scenario	Algorithm	Atmega128	MSP430	Ref.
Fixed key [9]	CHAM-64/128	27.9	49.3	This paper
	SPECK-64/128	29.8	50.0	[8]
	CHAM-128/128	17.1	25.0	This paper
	SPECK-128/128	12.7	21.7	[8]
Communication [26, 27] (without decryption)	CHAM-64/128	7.2	11.1	This paper
	SPECK-64/128	6.3	9.7	FELICS website

2 Specifications and Design Principles

CHAM is a family of block ciphers with a 4-branch generalized Feistel structure. Each cipher is denoted by CHAM- n/k with a block size of n -bit and a key size of k -bit. Table 3 shows the list of ciphers in the family and their parameters. Here, r and w denote the number of rounds and the bit length of a branch (word), respectively.

Table 3. List of CHAM ciphers and their parameters.

Cipher	n	k	r	w	k/w
CHAM-64/128	64	128	80	16	8
CHAM-128/128	128	128	80	32	4
CHAM-128/256	128	256	96	32	8

CHAM consists of three algorithms with different parameters. All three algorithms in the family are suitable for resource-constrained environments, but at

the same time they have slightly different applications according to their parameters. CHAM-64/128 is more suitable for low-end devices, CHAM-128/128 is a better choice for general use on 32-bit microcontrollers, and CHAM-128/256 can serve where a higher security level is required.

2.1 Notations

We use the following notations to describe CHAM- n/k .

- $x\|y$: concatenation of bit strings x and y
- $x \boxplus y$: addition of x and y modulo 2^w
- $x \oplus y$: bit-wise exclusive OR (XOR) of x and y
- $\text{ROL}_i(x)$: rotation of w -bit word x to the left by i bits
- $w_H(x)$: Hamming weight of x , the number of nonzero bits in x .

2.2 Specifications

CHAM- n/k encrypts a plaintext $P \in \{0, 1\}^n$ to a ciphertext $C \in \{0, 1\}^n$ using a secret key $K \in \{0, 1\}^k$ by applying r iterations of the round function as follows.

Divide the plaintext P into four w -bit words $X_0[0]$, $X_0[1]$, $X_0[2]$, and $X_0[3]$, where $P = X_0[0]\|X_0[1]\|X_0[2]\|X_0[3]$. Next, compute the n -bit output $X_{i+1} = X_{i+1}[0]\|X_{i+1}[1]\|X_{i+1}[2]\|X_{i+1}[3]$ of the i -th round for $0 \leq i < r$ by

$$\begin{aligned} X_{i+1}[3] &\leftarrow \text{ROL}_8((X_i[0] \oplus i) \boxplus (\text{ROL}_1(X_i[1]) \oplus RK[i \bmod 2k/w])), \\ X_{i+1}[j] &\leftarrow X_i[j+1] \text{ for } 0 \leq j \leq 2, \end{aligned}$$

if i is even, otherwise,

$$\begin{aligned} X_{i+1}[3] &\leftarrow \text{ROL}_1((X_i[0] \oplus i) \boxplus (\text{ROL}_8(X_i[1]) \oplus RK[i \bmod 2k/w])), \\ X_{i+1}[j] &\leftarrow X_i[j+1] \text{ for } 0 \leq j \leq 2, \end{aligned}$$

where $RK[i \bmod 2k/w]$ is the round key.

Then, the ciphertext C is defined by $C = X_r[0]\|X_r[1]\|X_r[2]\|X_r[3]$.

The key schedule of CHAM- n/k takes the secret key $K \in \{0, 1\}^k$ and generates the $2k/w$ w -bit round keys $RK[0], RK[1], \dots, RK[2k/w - 1]$. Initially, divide the secret key into k/w w -bit words $K[0], K[1], \dots, K[k/w - 1]$, where $K = K[0]\|K[1]\|\dots\|K[k/w - 1]$. Then the round keys are generated as follows.

$$\begin{aligned} RK[i] &\leftarrow K[i] \oplus \text{ROL}_1(K[i]) \oplus \text{ROL}_8(K[i]), \\ RK[(i + k/w) \oplus 1] &\leftarrow K[i] \oplus \text{ROL}_1(K[i]) \oplus \text{ROL}_{11}(K[i]), \end{aligned}$$

where $0 \leq i < k/w$.

The structures of the key schedule and round function of CHAM are depicted in Fig. 1, and the test vectors are provided in Appendix A.

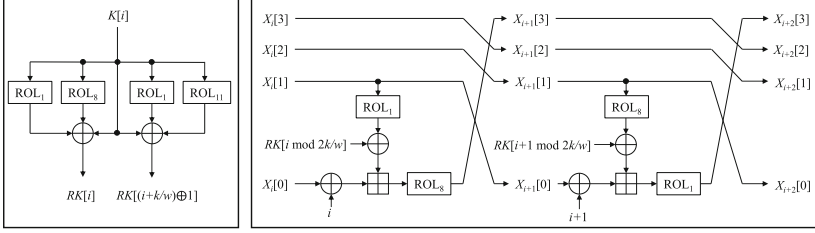


Fig. 1. Key schedule (left) and two consecutive round functions beginning with the even i -th round (right).

2.3 Design Principles

Design Paradigm and Design Approach. Our goal is to design a cipher that shows better efficiency than SIMON and SPECK on both (resource-constrained) hardware and software. So we design in the ARX design paradigm so that our cipher could have advantages in various resource-constrained environments. However, it is still not easy to prove nor to theoretically bound the minimum number of rounds with respect to the security of an ARX block cipher (although there is an exception [28]). Hence, we decided to design our algorithms (round function and key schedule) with a focus on efficiency and lightweight aspect and to perform a comprehensive and detailed cryptanalysis relying on experimental methods.

Round Functions. In the instruction sets of AVR and MSP introduced in Sect. 4, there is no single instruction for a rotation of arbitrary amounts. Only byte-swap and 1-bit rotation can be used for implementing $ROL_i()$. So, if we want to implement 3-bit rotation, we have to implement 1-bit rotation for 3 times. Hence, we choose every rotation amount as close as possible to 0 or 8 for the round functions (as well as the key schedule) to achieve a better performance. At the same time, in order to reduce the number of rounds while keeping security, we select several combinations of rotation amounts for a round function and consider combinations of the round functions for the whole encryption.

At first, we compare several cases of using a single round function and those of using two slightly different round functions. Let's denote rotation amounts of two consecutive rounds as a 4-tuple (a, b, c, d) such that a and c are rotation amounts for the values before the round key XOR and b and d are rotation amounts for outputs of the additions. For example, CHAM is of type $(1, 8, 8, 1)$. We estimate the maximum numbers of rounds of differential characteristics with probabilities greater than 2^{-n} when $w = 16$. Table 4 shows the results.

In Table 4, the type $(1, 8, 9, 1)$ looks like the best choice, however the type $(1, 8, 9, 1)$ requires more operations than the type $(1, 8, 8, 1)$ for every two rounds, so the overall efficiency of the type $(1, 8, 8, 1)$ is better. Also, we test the case of using 4-round alternating structures but could not find any better case than our selection in the overall efficiency point of view.

Table 4. Security evaluation results of several types of round functions.

Type	(0, 9, 0, 9)	(1, 8, 1, 8)	(1, 8, 1, 9)	(1, 8, 8, 1)	(1, 8, 9, 1)
Diff. ch. round	58	47	40	36	33

Additionally, we use round indexes as round constants not only to defend the slide attack and the rotational attack and but also to reduce the extra registers for storing constants on both hardware and software implementations.

Key Schedule. Key schedule is a procedure to calculate round keys from a given secret key. Many block ciphers (AES, SIMON/SPECK [8], and so on) have their own key schedules that consist of functions calculating each round keys from the current key state and update the key state. And key schedules of some other ciphers (PICCOLO [43], LED [29], and so on) consist of functions which calculate every round key directly from the secret key. Although the former key schedules could be also implemented to calculate every round key directly from the secret key, the latter key schedules are more suitable for such implementation. We call this implementation method as *stateless-on-the-fly* implementation and a key schedule like latter ones as *stateless-on-the-fly* key schedule.

It is clear that a stateless-on-the-fly implementation of a key schedule does not require areas (flip-flops) for storing key states on hardware, so it can be replaced by some selecting logics. For a k -bit secret key, at least $k \times s_f$ gates are required for storing key state while roughly at most $(k - 1) \times s_m$ gates are needed for some selecting logic, where s_f and s_m are gate counts of D flip-flop and 2-to-1 MUX, respectively. Therefore, the stateless-on-the-fly implementation helps to save the hardware areas when $s_m < s_f$. For an example of CHAM-64/128 in IBM130 [8], we can theoretically save 259 gates by adopting stateless-on-the-fly implementation for serial implementation, since $s_m = 2.25$ and $s_f = 4.25$. In practice, more gates could be saved for stateless-on-the-fly implementation because compiler could employ 3-to-1 or 4-to-1 MUXes instead of 2-to-1 MUXes to optimize the area while D flip-flop cannot be replaced by the other cell.

The key schedules of CHAM consist of a linear function Φ from $\{0, 1\}^w$ to $\{0, 1\}^{2w}$. Let I_w be the $w \times w$ identity matrix and I_w^l be a $w \times w$ matrix whose i -th row is defined by l times left rotation of i -th row of I_w for $i = 1, 2, \dots, w$. Then, the function Φ can be expressed by multiplication with the following binary matrix A defined by

$$A = (A_1 | A_2)^T = (I_w \oplus I_w^1 \oplus I_w^8 \mid I_w \oplus I_w^1 \oplus I_w^{11})^T.$$

Let $\mathcal{A}$ be a set of all matrices $(I_w^{l_0} \oplus I_w^{l_1} \oplus I_w^{l_2} \mid I_w^{l_3} \oplus I_w^{l_4} \oplus I_w^{l_5})^T$, where $l_0 = l_3 = 0, l_0 \neq l_1 \neq l_2, l_3 \neq l_4 \neq l_5$ and $0 < l_1, l_2, l_4, l_5 < w$. Let

$$\mathcal{A}_j = \{M \in \mathcal{A} \mid \min_{x \in \{0, 1\}^w \setminus \{0\}} (w_H(M \cdot x)) = j\}.$$

We make certain that in the case of $w = 16$, $\mathcal{A} = \mathcal{A}_2 \cup \mathcal{A}_4 \cup \mathcal{A}_6$.

The matrix A is the case of $l_0 = 0$, $l_1 = 1$, $l_2 = 8$, $l_3 = 0$, $l_4 = 1$, and $l_5 = 11$. It is chosen from $\mathcal{A}_6$ considering the number of instructions for implementations on AVR and MSP and cryptographic properties in the related-key model. Note that the matrix with $l_0 = 0$, $l_1 = 1$, $l_2 = 8$, $l_3 = 0$, $l_4 = 1$, and $l_5 = 9$ is in $\mathcal{A}_4$ and shows worse property for the related-key differential cryptanalysis. And the matrix with $l_0 = 0$, $l_1 = 1$, $l_2 = 8$, $l_3 = 0$, $l_4 = 1$, and $l_5 = 10$ also shows worse property than A in the viewpoint of the related-key differential cryptanalysis, even though it belongs to $\mathcal{A}_6$.

Additionally, we let the round keys be iteratively re-applied to reduce the size of memory for storing the round keys in software. And we give the round key index a tweak so that a w -bit word of a secret key cannot regularly act on every k/w rounds.

3 Hardware Implementation

We implemented CHAM in Verilog and synthesized using Synopsys Design Compiler 2008.09-SP5 without the *compile_ultra* feature. For the synthesis, the 100-kHz frequency constraint is imposed. Clocks for plaintext flip-flops are gated and scan flip-flops are not used. To evaluate and compare the hardware cost and performance, we use the UMC 90 nm, UMC 180 nm CMOS generic process library (UMC90, UMC180) provided by the Faraday Technology Corporation and IBM's 8RF 130 nm process (IBM130).

Figures 2 and 3 present the round-based and the bit-serial hardware architecture of CHAM, respectively. In both architectures, there are no flip-flops for

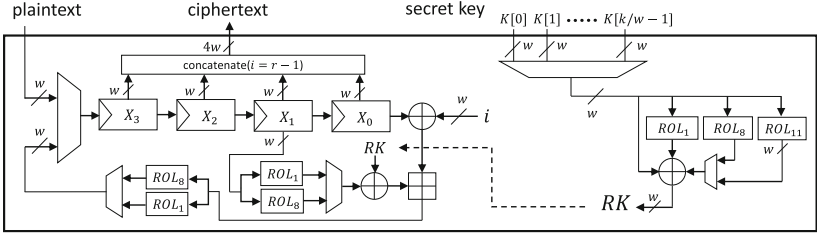


Fig. 2. Round-based hardware architecture of CHAM.

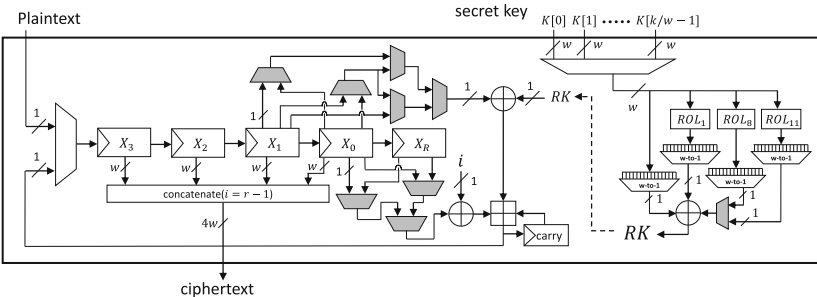


Fig. 3. Serialized hardware architecture of CHAM.

storing the secret key because the secret key is injected for each clock count. Instead of the flip-flops, a k/w -to-1 w -bit MUX is required for the round-based architecture, and a k -to-1 1-bit MUX is required for the bit-serial architecture. Since the size of the flip-flops is larger than that of the MUXes, we can save as much as the difference between the two sizes.

Table 5 shows the results of comparison to other lightweight block ciphers for each parameter and architecture in area (GE, gate equivalent) and throughput

Table 5. Comparison of the hardware implementation of CHAM and other ciphers.

n	k	Cipher	Bit-serial		Round-based		Tech.	Ref.
			Area (GE)	Throughput	Area (GE)	Throughput		
64	128	Twine	–	–	1,866	178.0	90n	[47]
		Hight	–	–	3,048	188.3	250n	[31]
		Gift	930	–	1,345	–	STM90	[4]
		Midori	–	–	1,542	–	STM90	[3]
		Midori	–	–	1,638	–	STM65	[3]
		Skinny	1,399	8.1	1,696	177.8	UMC180	[11]
		LED	1,265	3.4	–	–	–	[29]
		†LED	700	3.4	–	–	–	[29]
		Present	1,391	11.4	1,884	200.0	UMC180	[42]
		Piccolo	758	12.1	1,197	193.9	130n	[43]
		Simon	944	4.2	1,403	133.3	‡IBM130	[55]
		Simeck	924	4.2	1,365	133.3	‡IBM130	[55]
		Simon	958	4.2	1,417	133.3	IBM130	[8]
		Speck	996	3.4	1,658	206.5	IBM130	[8]
		CHAM	665	5.0	826	80.0	IBM130	Ours
128	128	CHAM	859	5.0	1,110	80.0	UMC180	Ours
		CHAM	727	5.0	985	80.0	UMC90	Ours
	128	Gift	1,213	–	1,997	–	STM90	[4]
		Midori	–	–	2,522	–	STM90	[3]
		Midori	–	–	2,714	–	STM65	[3]
		Skinny	1,840	14.7	2,391	320.0	UMC180	[11]
		LEA	2,302	4.2	3,826	76.2	UMC130	[30]
		AES	–	–	2,400	57.0	UMC180	[40]
		Simon	1,234	2.9	2,090	182.9	IBM130	[8]
		Speck	1,280	3.0	2,727	376.5	IBM130	[8]
		CHAM	1,057	5.0	1,499	160.0	IBM130	Ours
		CHAM	1,296	5.0	1,899	160.0	UMC180	Ours
		CHAM	1,084	5.0	1,691	160.0	UMC90	Ours
	128	Skinny	2,655	12.3	3,312	266.7	UMC180	[11]
		Simon	1,782	2.6	2,776	168.4	IBM130	[8]
		Speck	1,840	2.8	3,284	336.8	IBM130	[8]
		CHAM	1,180	4.2	1,622	133.3	IBM130	Ours
		CHAM	1,481	4.2	2,087	133.3	UMC180	Ours
		CHAM	1,256	4.2	1,864	133.3	UMC90	Ours

†: Hard-wired key implementation.

‡: Not exactly same to the non-daggers IBM130.

(Kbps@100 KHz). As the table shows, every instance in our family can be implemented in much smaller area than SIMON with the corresponding parameter. Precisely, the area of the sequential logic for CHAM is much smaller than that of SIMON because the secret key is not stored in flip-flops. Moreover, in most cases (except for the round-based CHAM-64/128), the throughputs per area's are also better than those of SIMON.

4 Software Implementation

We compare software performances of our cipher and existing algorithms on three typical microcontrollers suitable for lightweight devices. In [10, 26, 53], Atmega128 (AVR), MSP430F1611 (MSP), and SAM3X8E (ARM) are widely used as target 8, 16, and 32-bit microcontrollers, respectively. We also choose them as our target platforms. Their features are briefly presented in Appendix B.1.

Software performances are measured based on two metrics, *rank* and FOM, under two usage scenarios, *Fixed-key* and *Communication*. The metrics and scenarios are introduced in [9, 26, 27], and described in Appendices B.3 and B.4.

All our software implementations of CHAM are done in assembly language to draw better performance as much as possible. Some of the ideas are presented in Appendix B.2. Note also that, in our work, all the best results of CHAM are obtained from 4-round unrolled implementations except for CHAM-128/128 on the ARM. In that case, 8-round unroll results the best FOM value.

Table 6 presents the results of a performance comparison on the AVR and MSP platforms using the rank metric for the fixed-key scenario. Either SPECK or CHAM is always ranked at the top on both AVR and MSP, for all of the

Table 6. Performance comparison using the rank metric on AVR and MSP, under the fixed-key scenario.

Size (n/k)	Algorithm	AVR				MSP			
		ROM	RAM	cpb	Rank	ROM	RAM	cpb	Rank
64/128	SPECK [10]	218	0	154	29.8	204	0	98	50.0
	CHAM	202	3	172	27.9	156	8	118	49.3
	SIMON [10]	290	0	253	13.6	280	0	177	20.2
	SPARX	448	2	224	9.9	366	14	136	18.7
	HIGHT [9]	336	0	311	9.6	—	—	—	—
128/128	CHAM	362	16	148	17.1	280	20	125	25.0
	SPECK [10]	460	0	171	12.7	438	0	105	21.7
	AES [10]	970	18	146	6.8	—	—	—	—
	LEA	754	17	203	6.3	646	24	147	9.8
128/256	CHAM	396	16	177	13.2	312	20	148	19.2
	SPECK [9]	476	0	181	11.6	—	—	—	—

parameters of the block size and key size. They outperform the remaining ciphers. The results for SPARX [28] and LEA are drawn from performance reports posted on the FELICS [26] project’s website.

Table 7 shows the performances based on the rank metric under the one-way communication scenario, emphasizing variable key encryption without decryption. The results of CHAM on the AVR and MSP, obtained by optimization¹ level two, show slightly better performance than that of SPECK. The ranks of SPECK are derived from the data reported in the FELICS website, and the best ones are shown in the table.

On the ARM platform, CHAM-64/128 shows relatively poor result due to its 16-bit word size.² On the other hand, CHAM-128/128, based on a 32-bit word size, presents relatively decent result compared to SPECK-64/128.³

Table 7. Performance comparison using the rank metric in the one-way communication scenario. EKS and Enc represent the key schedule and encryption.

Platform	Algorithm	ROM		RAM		Cycles		cpb	Rank
		EKS	Enc	Stack	Data	EKS	Enc		
AVR	CHAM-64/128	72	280	11	184	309	23,664	187	7.2
	SPECK-64/128	178	240	12	260	1,401	19,888	166	6.3
MSP	CHAM-64/128	68	210	16	184	275	16,715	133	11.1
	SPECK-64/128	126	180	16	260	1,242	14,155	120	9.7
ARM	SPECK-64/128	52	164	36	260	516	6,323	53	23.2
	CHAM-128/128	44	210	48	192	95	8,846	70	19.5
	CHAM-64/128	124	272	48	184	170	16,944	134	8.7

Table 8 shows performance comparison based on the FOM metric under the communication scenario. SPECK-64/128 reveals the best value, followed by CHAM-128/128.⁴ Considering its block size, CHAM-128/128 can be regarded as a very good performer. The FOM of CHAM-64/128 is degraded by relatively poor performance on ARM. However, due to the excellence on AVR and MSP, its FOM is still ahead of all other competitors, except for SPECK. Note that all the values other than CHAM are presented on the FELICS website.

¹ The FELICS platform provides a unified implementation environment which generates performance figures automatically. The FELICS software framework, written in C language, permits users to implement only the core parts of encryption, decryption and their key schedules. Due to the common operational C source codes, the performance results are affected by the compiler’s optimization option.

² A SIMD implementation might enhance performance. Since the ARMv7-M architecture provides very limited instructions for SIMD arithmetics, it seems to be very difficult to get non-trivial performance gain from SIMD approach.

³ The performance of SPECK-128/128 is not yet reported in the FELICS website.

⁴ In the comparison, we exclude Chaskey algorithm because it is not considered as a block cipher.

Table 8. Performance comparison using the FOM metric in the communication scenario. The key schedule, encryption and decryption steps are all included.

Algorithm	AVR			MSP			ARM			FOM
	ROM	RAM	cpb	ROM	RAM	cpb	ROM	RAM	cpb	
SPECK-64/128	874	302	351	572	296	253	444	308	129	5.00
CHAM-128/128	1,230	262	337	926	258	298	528	272	144	5.47
CHAM-64/128	844	225	394	578	220	290	606	244	319	6.52
RECTANGLE-64/128	1,118	353	506	844	402	361	654	432	289	7.80
SPARX-64/128	1,198	392	512	966	392	287	1,200	424	319	8.57

5 Security Analysis

In this section, we summarize the security analysis results of CHAM and describe how we determine the number of rounds for each algorithm. We then present detailed cryptanalysis results of a number of well-known attacks applicable to our ciphers.

5.1 Summary of the Security Analysis

Table 9 shows the maximum numbers of rounds of characteristics that can be used for each attack. Based on these results, we determine the total number of rounds of each cipher with the following considerations

- Round extension of characteristics
- Round extension during key-recovery phase
- Security margin

Table 9. The numbers of rounds of the best discovered characteristics for each cipher and cryptanalysis.

CHAM-	DC	RDC	LC	BC	RBC	IDC	ZCLC	DLC	RDLC	IC	RXDC
64/128	36	34	34	35	41	18	21	34	36	16	16
128/128	45	33	40	47	36	15	18	45	39	16	23
128/256	45	40	40	47	47	15	18	45	45	16	23

(R)DC: (Related-key)Differential Cryptanalysis, LC: Linear Cryptanalysis, (R)BC: (Related-key)Boomerang Cryptanalysis, IDC: Impossible Differential Cryptanalysis ZCLC: Zero-Correlation Linear Cryptanalysis, (R)DLC: (Related-key)Differential-Linear Cryptanalysis, IC: Integral Cryptanalysis, RXDC: Rotational-XOR-Differential Cryptanalysis.

By applying the probability gathering technique used in [45] to our differential cryptanalysis, an attacker may find differentials longer than the differential

characteristics in Table 10 in Appendix C.1. But we check experimentally and confirm that the attacker can have at most four rounds longer differentials. Also, we verified that a boomerang and a rectangle characteristics can also be extended at most four rounds despite their use of two differential characteristics. Although such extensions can be applied only to the cryptanalyses based on difference or linear approximation, we assume every type of characteristic can be extended by four rounds regarding this technique.

In the key-recovery phase of an attack, if $k = 2n$, then one can attack k/w more rounds by guessing $k/2$ bits of the secret key before filtering. Otherwise, one cannot utilize this type of round extension. After filtering, one can add at most three rounds during key guessing and determining steps. Hence, we claim that the maximal numbers of rounds for which one can mount a key-recovery attack are at most $4 + 2(k/w - 4) + 3$ more than the numbers of rounds in Table 9, regardless of actual characteristics.

To estimate the numbers of rounds conservatively, we assume that an attacker can mount successful attacks using the characteristics we have found, even if the actual attacks are infeasible. Finally, we determine the numbers of rounds (instances of r), as shown in Table 3 with these considerations together with a security margin greater than 30% for full rounds⁵.

5.2 Cryptanalysis Results

We cryptanalyze our ciphers with the following well-studied techniques, and the results are summarized in Table 9. We use state-of-the-art techniques, modified for ARX structure ciphers [7, 17, 45, 49]. Using these results, we estimate the necessary numbers of rounds to ensure proper security. This subsection includes the results of (RK⁶) Differential, Linear, and (RK) Boomerang cryptanalysis and other detailed cryptanalysis results are provided in Appendix C due to the page limit.

(RK) Differential and Linear Cryptanalysis. We searched for (RK) differential characteristics [16] and linear approximations of each cipher using an automated algorithm known as the *threshold search* suggested in [17]. It is considered as an appropriate variant of Matsui’s branch-and-bound algorithm [39] for ARX ciphers. Using this threshold search approach, we found differential characteristics with a probability of $p > 2^{-n}$ for each cipher. These results are given in Table 10. We denote a (RK) differential characteristic by (Δ_{key}) , $\Delta_{\text{in}} \rightarrow \Delta_{\text{out}}$, where Δ_{key} is the difference of the secret key, and Δ_{in} and Δ_{out} are the input and output differences, respectively.

For a linear cryptanalysis [38], we found linear approximations with bias $\epsilon > 2^{-n/2}$ for each cipher. These results are given in Table 11. Γ_{in} and Γ_{out} denote the input and output masks, respectively. We calculate the correlation

⁵ 30% is a relatively high ratio for a security margin compared to those associated with other ciphers.

⁶ RK stands for “related-key”.

of the linear approximation by determining the bias of every addition using Wallén’s formula [52] and applying Piling-Up Lemma [38].

(RK) Boomerang Cryptanalysis. The (RK) boomerang characteristics [15, 50] are constructed with two short (RK) differential characteristics. These characteristics are the most threatening ones for our ciphers. Examples of short (RK) differential characteristics for constructing (RK) boomerang characteristics are shown in Table 12. A (RK) boomerang characteristic is valid when $pq > 2^{-n/2}$, where p and q are the probabilities of the two differential characteristics ϕ_1 and ϕ_2 , respectively, used for constructing the (RK) boomerang characteristic. Note that (RK) rectangle cryptanalysis [13, 15] works using the same characteristics.

6 Conclusion

In this paper, we have presented CHAM, a family of lightweight block ciphers that supports widely used block/key lengths. It is suitable for highly resource-constrained devices, especially area-constrained hardware with the help of the *stateless-on-the-fly* key schedule. Efficiency evaluations and comparisons with other lightweight block ciphers on various platforms are provided to demonstrate the merit of our algorithms. A variety of security analyses with extensive searching have convinced us that it is secure against known attacks. We also believe that it will be resistant to future attacks due to its high security margins. Moreover, the simplicity and flexibility of CHAM allow one to design ciphers with other block sizes and key lengths. Of course, a rigorous security analysis should be followed.

A Test Vectors

Test vectors are represented in hexadecimal with the prefix ‘0x’.

CHAM-64/128

```
secret Key : 0x0100 0x0302 0x0504 0x0706 0x0908 0x0b0a 0x0d0c 0x0f0e
plaintext  : 0x1100 0x3322 0x5544 0x7766
ciphertext : 0x453c 0x63bc 0xdcfa 0xbf4e
```

CHAM-128/128

```
secret Key : 0x03020100 0x07060504 0x0b0a0908 0x0f0e0d0c
plaintext  : 0x33221100 0x77665544 0xbbaa9988 0xffeeddcc
ciphertext : 0xc3746034 0xb55700c5 0x8d64ec32 0x489332f7
```

CHAM-128/256

```
secret Key : 0x03020100 0x07060504 0x0b0a0908 0x0f0e0d0c
             0xf3f2f1f0 0xf7f6f5f4 0xfbfa9f98 0xfffffdff
plaintext  : 0x33221100 0x77665544 0xbbaa9988 0xffeeddcc
ciphertext : 0xa899c8a0 0xc929d55c 0xab670d38 0x0c4f7ac8
```

B Some Details About Software Implementation

B.1 Target Platforms

Atmega128 belongs to Atmel’s AVR microcontroller family with an 8-bit RISC architecture. It has 32 general-purpose registers and 133 instructions. It is equipped with 128 KBytes of flash and 4 KBytes of RAM. MSP430F1611, a microcontroller from Texas Instruments, adopts a 16-bit RISC architecture with 16 registers (12 of them are general-purpose registers) and 51 instructions, including the emulated ones. It has 48 KBytes of flash and 10 KBytes of RAM. The ARM Cortex-M3 is a 32-bit processor core based on the ARMv7-M architecture, with 12 general-purpose registers. The core is adopted in the Atmel SAM3X8E microcontroller installed on the well-known Arduino Due development board. SAM3X8E is equipped with 512 KBytes of flash and 96 KBytes of SRAM.

B.2 Implementating Bit-Wise Rotation

In ARX design like CHAM, rotations by certain bit sizes might be costly to implement on our 8 and 16-bit platforms. Efficient implementation of rotation is crucial to both high throughput and smaller memory. With this consideration, CHAM adopts ROL_8 , which can be performed for free on AVR platform.

The MSP430 provides a byte-swapping instruction, which is equivalent to ROL_8 for a 16-bit word. It is slightly tricky for a 32-bit word on the MSP430. Similarly to [20], ROL_8 can be carried out in seven instructions, as in Code 1 below.⁷ The code require an additional temporary register to hold a 16-bit data.

ARMv7-M provides a powerful instruction, barrel shifter, which can rotate a 32-bit word by any bit-size. Moreover, the instruction can perform a certain kind of operation additionally after the rotation. This fact gives rise to a good performance of CHAM-128/128. However, it appears that no single instruction of ARMv7-M can perform bit-wise rotation for a *16-bit* word. This explains the relatively low performance of CHAM-64/128 on ARMv7-M, as can be seen in Tables 7 and 8.

Code 1. MSP430 code for 8-bit left rotation; register pairs Rh and Rl store a 32-bit integer, ABCD, where each letter represents an 8-bit integer part; register Rt is temporary.

mov	Rl, Rt	:	Rh=(B,A),	Rl=(D,C),	Rt=(D,C)	ABCD
xor.b	Rh, Rl	:		Rl=(B^D,0)		
xor	Rh, Rl	:		Rl=(D,A)		
xor.b	Rt, Rh	:	Rh=(B^D,0)			
xor	Rt, Rh	:	Rh=(B,C)			
swpb	Rl	:		Rl=(A,D)		OODA
swpb	Rh	:	Rh=(C,B)			BC00

⁷ We implement ROR_8 , a right rotation for decryption, in eight instructions.

B.3 Performance Metrics

Lightweight IoT devices are usually considered to have constrained resources. This is why throughput alone does not fully describe the performance of an algorithm. A smaller code size and less RAM usage are also important factors to consider. In [9], the authors argue the same context and introduce the metric of *rank* as an overall performance indicator. It is defined as

$$\text{rank} = (10^6/\text{cpb})/(\text{ROM} + 2 \times \text{RAM}),$$

where *cpb* refers to the cycles per byte consumed for a task, and *ROM* and *RAM* are the byte sizes of the memory of each type. By definition, the larger rank is better. Note also that RAM is considered to be twice as costly as ROM.

The FOM (figure of merit) metric, defined recently in the FELICS [27], averages performances on AVR, MSP and ARM. For each implementation i on a device d , we measure memory usages $v_{\text{ROM}}^{i,d}$, $v_{\text{RAM}}^{i,d}$, and time cost $v_{\text{cost}}^{i,d}$. Among all the implementations of all the ciphers, the minimums of ROM, RAM and cost are also determined (possibly each from different implementations). Denote each minimum by m_{ROM}^d , m_{RAM}^d , and m_{cost}^d . Then, the performance parameter p_d for a cipher on a device d is defined by

$$p_d = \min_i \left(v_{\text{ROM}}^{i,d}/m_{\text{ROM}}^d + v_{\text{RAM}}^{i,d}/m_{\text{RAM}}^d + v_{\text{cost}}^{i,d}/m_{\text{cost}}^d \right).$$

Finally, the figure of merit for a cipher is defined by the average of three p_d 's,

$$\text{FOM} = (p_{\text{AVR}} + p_{\text{MSP}} + p_{\text{ARM}}) \times \frac{1}{3}.$$

The definition indicates the smaller FOM is better.⁸

B.4 Usage Scenarios

A block cipher suite usually consists of three distinctive algorithms: the key schedule, encryption and decryption. However, in lightweight applications, decryption tends to lose its role due to well-designed modes of operations for block ciphers. The combined performance of the key schedule and encryption is somewhat sensitive to their usage scenarios. For an easy comparison of our cipher with the results in the literature, we adopt two scenarios: simple encryption with a fixed key and data communication with variable keys.

Fixed-key scenario: In this scenario, a cipher is used for authenticating devices. There are no key schedules or decryption steps. Round keys are fixed in the device, i.e., specifically placed in the code area. Hence, their size is added to the code size. This scenario is used in Table 6.

⁸ It can be pointed out that the definition of the FOM has a drawback that whenever a new minimum is found by a better implementation of any cipher, the whole FOMs of all ciphers should be updated.

Communication scenario: In this scenario, a cipher is assumed to be used for data communication. It is defined as Scenario 1 in the FELICS [26]. Originally, the scenario contains encryption, decryption together with their key schedules, where 128 bytes of data are encrypted and decrypted in the CBC mode. Since encryption part is more important for lightweight application, we define *one-way* communication scenario by omitting the decryption part, which is used in Table 7. The scenario in its original meaning is also used in Table 8.

C Cryptanalysis Results

C.1 Tables of characteristics for (RK) Differential, Linear, and (RK) Boomerang Cryptanalysis

Tables 10, 11, and 12 show characteristics for (RK) Differential, Linear, and (RK) Boomerang Cryptanalysis, respectively.

Table 10. The best (RK) differential characteristics found. Only the input and output differences are shown. We assume that every operation is independent. The subscript x indicates a hexadecimal expression.

Model	n/k	Round/ p	Characteristic
Single-key model	64/128	$36/2^{-63}$	$(0004_x, 0408_x, 0A00_x, 0000_x) \rightarrow (0005_x, 8502_x, 0004_x, 0A00_x)$
	128/*	$45/2^{-125}$	$(01028008_x, 08200080_x, 04000040_x, 42040020_x) \rightarrow (00000000_x, 00110004_x, 04089102_x, 00080010_x)$
Related-key model	64/128	Key diff.	$(0000_x, 0000_x, 0000_x, 0000_x, 0040_x, 0000_x, 0000_x, 4000_x),$
		$34/2^{-61}$	$(20A0_x, 1050_x, 4000_x, 2020_x) \rightarrow (0141_x, 8080_x, 4841_x, 830A_x)$
	128/128	Key diff.	$(00000000_x, 00000000_x, 00000000_x, 40000000_x),$
		$33/2^{-125}$	$(00410500_x, 00210080_x, 80102000_x, 40002041_x) \rightarrow (12810001_x, 8A500940_x, 08001503_x, 06000220_x)$
	128/256	Key diff.	$(00000000_x, 00000000_x, 00000000_x, 00000000_x, 00000000_x, 40000000_x, 00000000_x, 00000000_x)$
		$40/2^{-127}$	$(42800040_x, 20010420_x, 00800104_x, 08408082_x) \rightarrow (04405080_x, 80040892_x, 01000010_x, 80a00008_x)$

Table 11. The best linear approximations found. We assume that every operation is independent.

n/k	Round	$\langle \Gamma_{in}, \Gamma_{out} \rangle$	ϵ
64/128	34	$\langle (0000_x, 1000_x, 10D9_x, 8C20_x), (CF06_x, 0202_x, 0100_x, 0009_x) \rangle$	2^{-31}
128/*	40	$\langle (00000000_x, 08001000_x, 40891038_x, 3000C800_x), (06082000_x, 00001001_x, 42040030_x, 20020000_x) \rangle$	2^{-63}

Table 12. Examples of the best (RK) boomerang characteristics that we found.

Model	n/k	Round/ p or q	Characteristic
Single-key model	64/128	$17/2^{-15}$	$(8200_x, 0100_x, 0001_x, 8000_x) \rightarrow (0400_x, 0004_x, 0502_x, 0088_x)$
		$18/2^{-16}$	$(8200_x, 0100_x, 0001_x, 8000_x) \rightarrow (0004_x, 0502_x, 0088_x, 0000_x)$
	128/128	$23/2^{-30}$	$(08104000_x, 04002000_x, 00000020_x, 00000010_x)$ $\rightarrow (40010000_x, 20408100_x, 00000001_x, 02000080_x)$
		$24/2^{-33}$	$(02000000_x, 41000000_x, 20410000_x, 10008000_x)$ $\rightarrow (00000000_x, 00040001_x, 81020400_x, 00000004_x)$
Related-key model	64/128	Key diff. $20/2^{-15}$	$(0000_x, 0000_x, 0080_x, 0000_x, 0000_x, 8000_x, 0000_x, 0000_x)$ $(0084_x, 0000_x, 0000_x, 0000_x) \rightarrow (0400_x, 0105_x, 8080_x, 0100_x)$
		Key diff. $21/2^{-16}$	$(0000_x, 0000_x, 0000_x, 0000_x, 0080_x, 0000_x, 0000_x, 8000_x)$ $(0801_x, 8400_x, 0084_x, 0000_x) \rightarrow (0000_x, 0400_x, 0105_x, 8080_x)$
	128/128	Key diff.	$(00000000_x, 00000000_x, 00000000_x, 00800000_x)$ $(00000801_x, 80000400_x, 00800004_x, 00000000_x)$ $\rightarrow (00020484_x, 01000005_x, 08020000_x, 04010A00_x)$
		Key diff. $18/2^{-31}$	$(00000000_x, 00000000_x, 00000000_x, 00800000_x)$ $(00000801_x, 80000400_x, 00800004_x, 00000000_x)$ $\rightarrow (00020484_x, 01000005_x, 08020000_x, 04010A00_x)$
		Key diff. $18/2^{-31}$	$(00000000_x, 00000000_x, 00000000_x, 00800000_x)$ $(00000801_x, 80000400_x, 00800004_x, 00000000_x)$ $\rightarrow (00020484_x, 01000005_x, 08020000_x, 04010A00_x)$
	128/256	Key diff.	$(00000000_x, 00000000_x, 00000000_x, 00000000_x)$ $(00000000_x, 40000000_x, 00000000_x, 00000000_x)$ $(08410002_x, 04008000_x, 00040080_x, 00020040_x)$ $\rightarrow (02020000_x, 01010240_x, 00000202_x, 04000004_x)$
		$23/2^{-27}$	$(00000000_x, 00000000_x, 00000000_x, 00000000_x)$ $(00000000_x, 40000000_x, 00000000_x, 00000000_x)$ $(08410002_x, 04008000_x, 00040080_x, 00020040_x)$ $\rightarrow (01010240_x, 00000202_x, 04000004_x, 02008002_x)$
		Key diff. $24/2^{-32}$	$(00000000_x, 00000000_x, 00000000_x, 00000000_x)$ $(00000000_x, 40000000_x, 00000000_x, 00000000_x)$ $(08410002_x, 04008000_x, 00040080_x, 00020040_x)$ $\rightarrow (01010240_x, 00000202_x, 04000004_x, 02008002_x)$

C.2 Impossible Differential Cryptanalysis and Zero-Correlation Linear Cryptanalysis

Impossible differential cryptanalysis [12] uses a differential characteristic that can never occur. A zero-correlation linear approximation [7] is the counter-part of the impossible differential characteristic in the linear cryptanalysis field. Examples of the best impossible differential characteristics and zero-correlation linear approximations as found here are given in Table 13.

C.3 (RK) Differential-Linear Cryptanalysis

A (RK) differential-linear approximation [14] is constructed with a short (RK) differential characteristic and a short linear approximation. A (RK) differential-linear approximation which has a correlation of $pc^2 > 2^{-n/2}$ can be used for a (RK) differential-linear attack, where p is the probability of the differential characteristic ϕ and c is the correlation of the linear approximation ψ . Examples showing how to build these (RK) differential-linear approximations are given in Table 14.

Table 13. Examples of the best impossible differential characteristics and zero correlation linear approximations found here.

Type	n/k	Round	Characteristic
Impossible differential	64/128	10	$(1^1 0^{15}, 0^{16}, 0^{16}, 0^{16}) \leftrightarrow (x^6 1^1 0^1 x^8, x^{14} 1^1 x^1, x^{16}, x^{13} 1^1 0^1 x^1)$
		8	$\nrightarrow (x^{16}, x^{16}, x^{16}, x^8 1^1 0^7) \leftrightarrow (0^8 1^1 0^7, 0^{16}, 0^{16}, 0^{15} 1^1)$
	128/*	8	$(0^{32}, 0^{32}, 0^{32}, 1^1 0^{31}) \leftrightarrow (0^{32}, x^{32}, x^{32}, x^{30} 1^1 x^1)$
		7	$\nrightarrow (x^{32}, x^{32}, x^{32}, x^{24} 1^1 0^7) \leftrightarrow (0^{32}, 0^{24} 1^1 0^7, 0^{32}, 0^{32})$
Zero correlation linear approximation	64/128	11	$(0^{15} 1^1, 1^1 0^{15}, 0^{16}, 0^{15} 1^1) \leftrightarrow (x^{16}, x^8 0^6 1^1 x^1, x^{16}, x^{16})$
		10	$\nrightarrow (1^1 x^{15}, x^8 1^1 x^7, x^{16}, x^{16}) \leftrightarrow (0^{13} 1^1 0^2, 0^{16}, 0^{16}, 0^{16})$
	128/*	9	$(0^{32}, 0^{32}, 0^{31} 1^1, 1^1 0^{31}) \leftrightarrow (x^{32}, 0^{15} 1^1 x^8 0^8, x^{32}, x^{32})$
		9	$\nrightarrow (0^{32}, 1^1 x^{31}, x^{32}, x^{32}) \leftrightarrow (0^{16} 1^1 0^{15}, 0^{32}, 0^{32}, 0^{32})$

Characteristics are denoted by 4-tuples of w -bit sequences separated by comma. 0^j , 1^j , and x^j are j -bit sequences of 0, 1, and free bit, respectively.

Table 14. Examples showing how to build the best (RK) differential-linear approximations found here.

Model	n/k	(RK) diff.-linear approx.		ϕ		ψ	
		Round	pc^2	Rounds	p	Round	c^2
Single-key model	64/128	34	2^{-31}	20	2^{-23}	14	2^{-8}
	128/*	45	2^{-63}	24	2^{-33}	21	2^{-30}
Related-key model	64/128	36	2^{-28}	22	2^{-18}	14	2^{-10}
	128/128	39	2^{-60}	17	2^{-26}	22	2^{-34}
	128/256	45	2^{-61}	23	2^{-27}	22	2^{-34}

C.4 Integral Cryptanalysis

Integral cryptanalysis [35] uses sets of chosen plaintexts of which a part is held constant and the other part varies through all possibilities. Considering ADD-balance [30], we found the following 16-round integral characteristic for all of our ciphers.

$$\begin{aligned}
& (A, C, C, C) \rightarrow (C, C, C, A) \rightarrow (C, C, A, C) \rightarrow (C, A, C, C) \rightarrow (A, C, C, A) \rightarrow (C, C, A, A) \rightarrow (C, A, A, C) \\
& \rightarrow (A, A, C, A) \rightarrow (A, C, A, B_{\lll 1}^+) \rightarrow (C, A, B_{\lll 1}^+, A) \rightarrow (A, B_{\lll 1}^+, A, A) \rightarrow (B_{\lll 1}^+, A, A, U) \\
& \rightarrow (A, A, U, U) \rightarrow (A, U, U, B_{\lll 8}^+) \rightarrow (U, U, B_{\lll 8}^+, U) \rightarrow (U, B_{\lll 8}^+, U, U) \rightarrow (B_{\lll 8}^+, U, U, U)
\end{aligned}$$

C , A , $B_{\lll l}^+$, U represent a constant word, an active word, an ADD-balanced word when rotated to the right by l bits, and an unknown word, respectively. The above 16-round distinguisher means that if the first word of a plaintext is active, which takes all w -bit values at one time, and the other words of the plaintext are constants, then the first word of the output after 16 rounds is ADD-balanced when rotated to the right by 8 bits.

The bit-based division property [49] is an improvement of the division property [48] for non S-box-based ciphers. In [46], Sun et al. improved the integral cryptanalysis result of LEA slightly by applying the bit-based division property. Based on this result and owing to the similarity between LEA and our ciphers, we expect that the bit-based division property will not seriously improve our integral cryptanalysis.

C.5 Biclique Cryptanalysis

Wang et al. [51] showed that for variants of the Feistel structure, interleaving related-key differential trails cannot construct bicliques [5]. Hence, we consider the bicliques from independent related-key differentials, as our ciphers have a variant of the type-3 generalized Feistel structure. We calculate the total complexity C_{full} for a key recovery attack with independent bicliques using the following equation,

$$C_{full} = 2^{k-2d}(C_{biclique} + C_{precomp} + C_{recomp}),$$

where $C_{biclique}$, $C_{precomp}$, and C_{recomp} denote the complexities for building-biclique, pre-computation, and re-computation, respectively. Note that a trivial biclique for each cipher can be derived easily from related-key differentials. The specific complexities are shown in Table 15. The re-check complexity of a false positive is omitted in the above equation because it is negligible.

Table 15. Complexity of the biclique cryptanalysis for each parameter.

n/k	Biclique round	$C_{biclique}$	$C_{precomp}$	C_{recomp}	C_{full}
64/128	15	$2^{14.6}$	$2^{16.6}$	$2^{31.5}$	$2^{127.5}$
128/128	7	$2^{29.5}$	$2^{32.7}$	$2^{63.7}$	$2^{127.7}$
128/256	15	$2^{30.3}$	$2^{32.6}$	$2^{63.6}$	$2^{255.6}$

C.6 Rotational Cryptanalysis

The initial version of a rotational cryptanalysis [33] can be easily defended by constant-XOR's. However, the recently proposed rotational-XOR cryptanalysis [2] can be well-applied to ARX ciphers with constant XOR's. So, we carefully applied the rotational-XOR cryptanalysis to our algorithm and the results are shown in the Table 16. Characteristics are initial and final δ 's. Refer to [2] for attack conditions and the definition of δ .

C.7 Other Attacks

Applying round constants keeps our ciphers secure against slide attacks [18]. An algebraic attack [24] is not effective for our ciphers due to the high nonlinearity of such a case.

Table 16. The best rotational-XOR differential characteristics found.

n/k	Round/ p	Characteristic (δ)
64/128	$16/2^{-62.225}$	$(0000_x, 0000_x, 0000_x, 8002_x) \rightarrow (0004_x, 020e_x, 0805_x, 0810_x)$
128/*	$23/2^{-125.545}$	$(10000004_x, 08000000_x, 02080003_x, 40040000_x)$ $\rightarrow (00100004_x, 08080400_x, 00220008_x, a1110d00_x)$

References

1. Albrecht, M.R., Driessen, B., Kavun, E.B., Leander, G., Paar, C., Yalçın, T.: Block ciphers – focus on the linear layer (feat. PRIDE). In: Garay, J.A., Gennaro, R. (eds.) CRYPTO 2014. LNCS, vol. 8616, pp. 57–76. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-662-44371-2_4
2. Ashur, T., Liu, Y.: Rotational cryptanalysis in the presence of constants. IACR Trans. Symmetric Cryptol. **2016**(1), 57–70 (2016)
3. Banik, S., Bogdanov, A., Isobe, T., Shibutani, K., Hiwatari, H., Akishita, T., Regazzoni, F.: Midori: a block cipher for low energy. In: Iwata, T., Cheon, J.H. (eds.) ASIACRYPT 2015. LNCS, vol. 9453, pp. 411–436. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48800-3_17
4. Banik, S., Pandey, S.K., Peyrin, T., Sasaki, Y., Sim, S.M., Todo, Y.: GIFT: a small present. In: Fischer, W., Homma, N. (eds.) CHES 2017. LNCS, vol. 10529, pp. 321–345. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-66787-4_16
5. Bogdanov, A., Khovratovich, D., Rechberger, C.: Biclique cryptanalysis of the full AES. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. LNCS, vol. 7073, pp. 344–371. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-25385-0_19
6. Bogdanov, A., Knudsen, L.R., Leander, G., Paar, C., Poschmann, A., Robshaw, M.J.B., Seurin, Y., Vikkelsøe, C.: PRESENT: an ultra-lightweight block cipher. In: Paillier, P., Verbauwheide, I. (eds.) CHES 2007. LNCS, vol. 4727, pp. 450–466. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-74735-2_31
7. Bogdanov, A., Rijmen, V.: Linear hulls with correlation zero and linear cryptanalysis of block ciphers. Des. Codes Cryptogr. **70**(3), 369–383 (2014). <https://doi.org/10.1007/s10623-012-9697-z>
8. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: The simon and speck families of lightweight block ciphers. IACR Cryptology ePrint Archive 2013, p. 404 (2013)
9. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: The SIMON and SPECK block ciphers on AVR 8-bit microcontrollers. In: Eisenbarth, T., Öztürk, E. (eds.) LightSec 2014. LNCS, vol. 8898, pp. 3–20. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-16363-5_1
10. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: Simon and speck: block ciphers for the internet of things. IACR Cryptology ePrint Archive 2015, p. 585 (2015)
11. Beierle, C., et al.: The SKINNY family of block ciphers and its low-latency variant MANTIS. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016. LNCS, vol. 9815, pp. 123–153. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53008-5_5

12. Biham, E., Biryukov, A., Shamir, A.: Cryptanalysis of skipjack reduced to 31 rounds using impossible differentials. In: Stern, J. (ed.) EUROCRYPT 1999. LNCS, vol. 1592, pp. 12–23. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48910-X_2
13. Biham, E., Dunkelman, O., Keller, N.: The rectangle attack — rectangling the serpent. In: Pfitzmann, B. (ed.) EUROCRYPT 2001. LNCS, vol. 2045, pp. 340–357. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-44987-6_21
14. Biham, E., Dunkelman, O., Keller, N.: Enhancing differential-linear cryptanalysis. In: Zheng, Y. (ed.) ASIACRYPT 2002. LNCS, vol. 2501, pp. 254–266. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-36178-2_16
15. Biham, E., Dunkelman, O., Keller, N.: Related-key boomerang and rectangle attacks. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 507–525. Springer, Heidelberg (2005). https://doi.org/10.1007/11426639_30
16. Biham, E., Shamir, A.: Differential Cryptanalysis of the Data Encryption Standard. Springer, New York (1993). <https://doi.org/10.1007/978-1-4613-9314-6>. ISBN: 978-1-4613-9316-0, 978-1-4613-9314-6
17. Biryukov, A., Velichkov, V.: Automatic search for differential trails in ARX ciphers. In: Benaloh, J. (ed.) CT-RSA 2014. LNCS, vol. 8366, pp. 227–250. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-04852-9_12
18. Biryukov, A., Wagner, D.: Slide attacks. In: Knudsen, L. (ed.) FSE 1999. LNCS, vol. 1636, pp. 245–259. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48519-8_18
19. Borghoff, J., et al.: PRINCE – a low-latency block cipher for pervasive computing applications. In: Wang, X., Sako, K. (eds.) ASIACRYPT 2012. LNCS, vol. 7658, pp. 208–225. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-34961-4_14
20. Buhrow, B., Riemer, P., Shea, M., Gilbert, B., Daniel, E.: Block cipher speed and energy efficiency records on the MSP430: system design trade-offs for 16-bit embedded applications. In: Aranha, D.F., Menezes, A. (eds.) LATINCRYPT 2014. LNCS, vol. 8895, pp. 104–123. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-16295-9_6
21. De Cannière, C., Dunkelman, O., Knežević, M.: KATAN and KTANTAN — a family of small and efficient hardware-oriented block ciphers. In: Clavier, C., Gaj, K. (eds.) CHES 2009. LNCS, vol. 5747, pp. 272–288. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-04138-9_20
22. Canteaut, A., Lallemand, V., Naya-Plasencia, M.: Related-key attack on full-round PICARO. In: Dunkelman, O., Keliher, L. (eds.) SAC 2015. LNCS, vol. 9566, pp. 86–101. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-31301-6_5
23. Chen, J., Teh, J.S., Su, C., Samsudin, A., Fang, J.: Improved (related-key) attacks on round-reduced KATAN-32/48/64 based on the extended boomerang framework. In: Liu, J.K., Steinfeld, R. (eds.) ACISP 2016. LNCS, vol. 9723, pp. 333–346. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-40367-0_21
24. Courtois, N.T., Pieprzyk, J.: Cryptanalysis of block ciphers with overdefined systems of equations. In: Zheng, Y. (ed.) ASIACRYPT 2002. LNCS, vol. 2501, pp. 267–287. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-36178-2_17
25. Dai, Y., Chen, S.: Cryptanalysis of full PRIDE block cipher. Sci. China Inf. Sci. **60**, 052108 (2017). <https://doi.org/10.1007/s11432-015-5487-3>
26. Dinu, D., Biryukov, A., Großschädl, J., Khovratovich, D., Le Corre, Y., Perrin, L.: FELICS - fair evaluation of lightweight cryptographic systems. In: NIST Workshop on Lightweight Cryptography 2015 National Institute of Standards and Technology (2015)

27. Dinu, D., Le Corre, Y., Khovratovich, D., Perrin, L., Großschädl, J., Biryukov, A.: Triathlon of lightweight block ciphers for the Internet of things. IACR Cryptology ePrint Archive, p. 209 (2015)
28. Dinu, D., Perrin, L., Udovenko, A., Velichkov, V., Großschädl, J., Biryukov, A.: Design strategies for ARX with provable bounds: SPARX and LAX. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016. LNCS, vol. 10031, pp. 484–513. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53887-6_18
29. Guo, J., Peyrin, T., Poschmann, A., Robshaw, M.: The LED block cipher. In: Preneel, B., Takagi, T. (eds.) CHES 2011. LNCS, vol. 6917, pp. 326–341. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-23951-9_22
30. Hong, D., Lee, J.-K., Kim, D.-C., Kwon, D., Ryu, K.H., Lee, D.-G.: LEA: a 128-bit block cipher for fast encryption on common processors. In: Kim, Y., Lee, H., Perrig, A. (eds.) WISA 2013. LNCS, vol. 8267, pp. 3–27. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-05149-9_1
31. Hong, D., et al.: HIGHT: a new block cipher suitable for low-resource device. In: Goubin, L., Matsui, M. (eds.) CHES 2006. LNCS, vol. 4249, pp. 46–59. Springer, Heidelberg (2006). https://doi.org/10.1007/11894063_4
32. Jean, J., Nikolić, I., Peyrin, T., Wang, L., Wu, S.: Security analysis of PRINCE. In: Moriai, S. (ed.) FSE 2013. LNCS, vol. 8424, pp. 92–111. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-662-43933-3_6
33. Khovratovich, D., Nikolić, I.: Rotational cryptanalysis of ARX. In: Hong, S., Iwata, T. (eds.) FSE 2010. LNCS, vol. 6147, pp. 333–346. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-13858-4_19
34. Knudsen, L., Leander, G., Poschmann, A., Robshaw, M.J.B.: PRINTCIPHER: a block cipher for IC-printing. In: Mangard, S., Standaert, F.-X. (eds.) CHES 2010. LNCS, vol. 6225, pp. 16–32. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15031-9_2
35. Knudsen, L., Wagner, D.: Integral cryptanalysis. In: Daemen, J., Rijmen, V. (eds.) FSE 2002. LNCS, vol. 2365, pp. 112–127. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45661-9_9
36. Kolay, S., Mukhopadhyay, D.: Khudra: a new lightweight block cipher for FPGAs. In: Chakraborty, R.S., Matyas, V., Schaumont, P. (eds.) SPACE 2014. LNCS, vol. 8804, pp. 126–145. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-12060-7_9
37. Koo, B., Hong, D., Kwon, D.: Related-key attack on the full HIGHT. In: Rhee, K.-H., Nyang, D.H. (eds.) ICISC 2010. LNCS, vol. 6829, pp. 49–67. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-24209-0_4
38. Matsui, M.: Linear cryptanalysis method for DES cipher. In: Hellese, T. (ed.) EUROCRYPT 1993. LNCS, vol. 765, pp. 386–397. Springer, Heidelberg (1994). https://doi.org/10.1007/3-540-48285-7_33
39. Matsui, M.: On correlation between the order of S-boxes and the strength of DES. In: De Santis, A. (ed.) EUROCRYPT 1994. LNCS, vol. 950, pp. 366–375. Springer, Heidelberg (1995). <https://doi.org/10.1007/BFb0053451>
40. Moradi, A., Poschmann, A., Ling, S., Paar, C., Wang, H.: Pushing the limits: a very compact and a threshold implementation of AES. In: Paterson, K.G. (ed.) EUROCRYPT 2011. LNCS, vol. 6632, pp. 69–88. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-20465-4_6
41. Piret, G., Roche, T., Carlet, C.: PICARO – a block cipher allowing efficient higher-order side-channel resistance. In: Bao, F., Samarati, P., Zhou, J. (eds.) ACNS 2012. LNCS, vol. 7341, pp. 311–328. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-31284-7_19

42. Poschmann, A.: Lightweight cryptography - cryptographic engineering for a pervasive world. Number 8 in IT Security. Europäischer Universitätsverlag, Published: Ph.D. thesis, Ruhr University Bochum (2009)
43. Shibutani, K., Isobe, T., Hiwatari, H., Mitsuda, A., Akishita, T., Shirai, T.: *Piccolo*: an ultra-lightweight blockcipher. In: Preneel, B., Takagi, T. (eds.) CHES 2011. LNCS, vol. 6917, pp. 342–357. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-23951-9_23
44. Shirai, T., Shibutani, K., Akishita, T., Moriai, S., Iwata, T.: The 128-bit blockcipher CLEFIA (extended abstract). In: Biryukov, A. (ed.) FSE 2007. LNCS, vol. 4593, pp. 181–195. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-74619-5_12
45. Song, L., Huang, Z., Yang, Q.: Automatic differential analysis of ARX block ciphers with application to SPECK and LEA. In: Liu, J.K., Steinfeld, R. (eds.) ACISP 2016. LNCS, vol. 9723, pp. 379–394. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-40367-0_24
46. Sun, L., Wang, W., Liu, R., Wang, M.: MILP-aided bit-based division property for ARX-based block cipher, Cryptology ePrint Archive, Report 2016, p. 1101 (2016)
47. Suzaki, T., Minematsu, K., Morioka, S., Kobayashi, E.: *TWINE*: a lightweight block cipher for multiple platforms. In: Knudsen, L.R., Wu, H. (eds.) SAC 2012. LNCS, vol. 7707, pp. 339–354. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-35999-6_22
48. Todo, Y.: Integral cryptanalysis on full MISTY1. In: Gennaro, R., Robshaw, M. (eds.) CRYPTO 2015. LNCS, vol. 9215, pp. 413–432. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-47989-6_20
49. Todo, Y., Morii, M.: Bit-based division property and application to SIMON family. In: Peyrin, T. (ed.) FSE 2016. LNCS, vol. 9783, pp. 357–377. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-52993-5_18
50. Wagner, D.: The boomerang attack. In: Knudsen, L. (ed.) FSE 1999. LNCS, vol. 1636, pp. 156–170. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48519-8_12
51. Wang, Y., Wu, W., Yu, X., Zhang, L.: Security on LBlock against biclique cryptanalysis. In: Lee, D.H., Yung, M. (eds.) WISA 2012. LNCS, vol. 7690, pp. 1–14. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-35416-8_1
52. Wallén, J.: On the differential and linear properties of addition master’s thesis. Helsinki University of Technology, Laboratory for Theoretical Computer Science (2003)
53. Wenzel-Benner, C., Gräf, J.: XBX: eXternal benchmarking eXtension for the SUPERCOP crypto benchmarking framework. In: Mangard, S., Standaert, F.-X. (eds.) CHES 2010. LNCS, vol. 6225, pp. 294–305. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15031-9_20
54. Yang, Q., Hu, L., Sun, S., Song, L.: Related-key impossible differential analysis of full *Khudra*. In: Ogawa, K., Yoshioka, K. (eds.) IWSEC 2016. LNCS, vol. 9836, pp. 135–146. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-44524-3_8
55. Yang, G., Zhu, B., Suder, V., Aagaard, M.D., Gong, G.: The Simeck family of lightweight block ciphers. In: Güneysu, T., Handschuh, H. (eds.) CHES 2015. LNCS, vol. 9293, pp. 307–329. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48324-4_16